

Serverless Personal To-do List Application

By

Name: Mathamsetty Sri Giri Naga Surya Vaibhav

Domain: Cloud Computing

Registered Email: mvaibhav5874@gmail.com

ABSTRACT

In the evolving landscape of cloud computing, serverless architecture is gaining popularity due to its scalability, cost-efficiency, and ease of deployment. This project titled "Serverless Personal To-Do List Application" aims to design and implement a task management app entirely on Amazon Web Services (AWS) using a serverless approach. By removing the overhead of server management, developers can focus solely on application functionality and business logic.

The application allows users to Create, Read, Update, and Delete (CRUD) to-do items through a RESTful interface. It is built using AWS Lambda for backend logic, Amazon API Gateway to expose endpoints, and Amazon DynamoDB for persistent storage. The entire infrastructure is defined and deployed using AWS SAM (Serverless Application Model), simplifying DevOps and ensuring version-controlled, repeatable deployments. This project provides an excellent learning experience in building practical, cloud-native applications using AWS services.

OBJECTIVE

The primary objective of this project is to **build a functional and fully serverless personal to-do list web service**. The sub-goals include:

- Understanding and implementing **Lambda functions** for application logic.
- Designing **RESTful APIs** using **Amazon API Gateway**.
- Storing task-related data securely in a **DynamoDB** table.
- Ensuring seamless deployment and updates using **AWS SAM**.
- Gaining experience with real-world use of **CRUD operations** in serverless architecture.
- Hosting a cost-effective application that can scale on-demand without infrastructure management.

This project emphasizes the benefits of serverless computing, hands-on experience with AWS tools, and practical implementation of cloud-native principles.

INTRODUCTION

In the rapidly evolving world of software development, businesses and developers are increasingly adopting **cloud-native and serverless architectures** to reduce complexity, minimize costs, and improve scalability. Traditionally, deploying an application involved significant effort in provisioning, configuring, and maintaining servers, setting up network routing, scaling infrastructure based on demand, and ensuring uptime and security. This operational overhead often diverted the focus away from actual application development and innovation.

Serverless computing addresses this challenge by abstracting away server management. Developers no longer have to worry about infrastructure provisioning, scaling, patching, or availability. Instead, they can simply write and deploy their code, and the cloud provider automatically handles everything else—from scaling to managing uptime. Serverless architecture enables faster development cycles, lower costs, and increased focus on delivering business value.

The Serverless Paradigm and AWS

Among the cloud providers, **Amazon Web Services (AWS)** has established itself as a leader in serverless computing. AWS offers a comprehensive and mature ecosystem that empowers developers to build production-grade applications without managing underlying infrastructure. This project leverages a combination of AWS's serverless tools to develop a **To-Do List application**, a simple yet effective demonstration of modern cloud-native design.

Key AWS Services Used

1. AWS Lambda

AWS Lambda is the core compute service in AWS's serverless offering. It allows developers to run backend code in response to events (such as HTTP requests) without provisioning or managing servers. In this project, AWS Lambda handles the business logic for all CRUD (Create, Read, Update, Delete) operations. Each function is event-driven and executes only when triggered by an API call, making it extremely cost-efficient.

Benefits of using AWS Lambda:

- **Automatic scaling:** Lambda functions automatically handle any number of concurrent requests.
- **Pay-as-you-go pricing:** You are only billed for the compute time you consume.

- **No server management:** Complete abstraction from server-level concerns such as OS maintenance or scaling.

2. Amazon API Gateway

Amazon API Gateway is a fully managed service that helps developers create, publish, maintain, monitor, and secure RESTful APIs. It acts as the front door to the backend logic hosted in AWS Lambda. In this project, API Gateway:

- Exposes REST endpoints such as /todos, /todos/{id}.
- Validates and routes incoming HTTP requests.
- Integrates securely with Lambda functions.
- Enables throttling, authorization, and usage plans.

By using API Gateway, the application benefits from **a highly available, low-latency API layer** without the need to deploy or manage web servers.

3. Amazon DynamoDB

For data persistence, the project uses **Amazon DynamoDB**, a fast and flexible NoSQL database service designed for single-digit millisecond performance. DynamoDB is fully managed and automatically scales to handle millions of requests per second.

In the context of this application:

- To-Do items are stored as individual records (items) in a DynamoDB table.
- Each item includes attributes such as id, task, status, and timestamp.
- DynamoDB's key-value model is well-suited for task management use cases.

Using DynamoDB ensures **low-latency read/write operations**, seamless scaling, and zero maintenance of database servers.

4. AWS SAM (Serverless Application Model)

AWS SAM is an open-source framework that simplifies the process of developing, testing, and deploying serverless applications. It uses a **YAML-based template** to define cloud resources such as functions, APIs, and databases.

In this project, SAM:

- Defines the entire infrastructure as code (template.yaml).

- Manages permissions, API configurations, environment variables, and function properties.
- Enables local testing and fast deployments using `sam build` and `sam deploy`.

SAM brings **consistency, repeatability, and version control** to infrastructure management, which aligns with modern DevOps and Infrastructure-as-Code (IaC) practices.

Purpose and Relevance of the Application

The **Serverless Personal To-Do List Application** aims to provide users with a convenient and efficient way to track and manage their daily tasks. While the functionality of a To-Do list may seem simple, it serves as an ideal use case for demonstrating the **core principles of serverless computing**. It offers hands-on exposure to creating RESTful APIs, managing data persistence, and deploying scalable backend logic—skills that are directly applicable to real-world software development.

The application architecture is **highly scalable, cost-effective, and secure**:

- It can support a single user or thousands without infrastructure modification.
- There is no idle resource cost, as all services are billed based on usage.
- Security and access are controlled through AWS Identity and Access Management (IAM) and API Gateway policies.

METHODOLOGY

The development of this application followed a step-by-step approach:

1. Environment Setup

- Installed **AWS CLI**, **AWS SAM CLI**, and configured IAM user credentials.
- Created a new SAM application using : `sam init --runtime nodejs18.x --name todo-app`

2. DynamoDB Table Setup

- Created a table named `TodoTable` with `id` as the **Primary Key**.
- Configured IAM permissions to allow Lambda functions to interact with DynamoDB.

3. Developing the Lambda Function

- Wrote the backend logic using **Node.js** to perform all CRUD operations.
- Used AWS SDK (`aws-sdk` package) to interact with DynamoDB.
- The function supports:
 - POST - Create a new to-do item.
 - GET - Retrieve all to-do items.
 - PUT - Update an existing item.
 - DELETE - Remove an item.

4. API Gateway Configuration

- Defined API endpoints in the `template.yaml` file.
- Mapped HTTP methods and routes to the Lambda function.

5. Deployment Using AWS SAM

- Used `sam build` to compile the application.
- Deployed using : `sam deploy --guided`
- Verified deployment through the AWS Console and API Gateway URL.

6. Testing the Application

- Used **Postman** to test all four operations by hitting the respective endpoints.
- Verified the output in **CloudWatch Logs** and **DynamoDB** entries.

CODE & RESULTS (Brief Overview)

Lambda Function (Node.js sample snippet):

js

CopyEdit

```
const AWS = require('aws-sdk');

const dynamo = new AWS.DynamoDB.DocumentClient();

exports.handler = async (event) => {

  const method = event.httpMethod;

  if (method === 'POST') {

    // create logic

  } else if (method === 'GET') {

    // read logic

  } else if (method === 'PUT') {

    // update logic

  } else if (method === 'DELETE') {

    // delete logic

  }

};
```

template.yaml (sample portion):

yaml

CopyEdit

Resources:

 TodoFunction:

 Type: AWS::Serverless::Function

 Properties:

 Handler: app.handler

 Runtime: nodejs18.x

 Events:

 TodoApi:

 Type: Api

Properties:

Path: /todo

Method: ANY

The Pulumi code is designed to deploy into different environments, such as production and development and the config file for dev is in **Pulumi.dev.yaml**.

The code for the Pulumi infrastructure resource is:

```
import
pulumi,
json

import pulumi_aws as aws

from pulumi_docker import Image, DockerBuild

import pulumi_docker as docker

from pulumi import Config

# Create a config object to access configuration values
config = pulumi.Config()

docker_image = config.get("docker_image")
environment = config.get("environment")
region = config.get("region")

aws.config.region = region

# First, create the DynamoDB table with just `id` as the primary key
```

```

dynamodb_table = aws.dynamodb.Table(

    f"todo-{environment}",

    name=f"todo-{environment}",

    hash_key="id", # Only `id` as the partition key

    attributes=[

        aws.dynamodb.TableAttributeArgs(

            name="id",

            type="S" # `S` for string type (use appropriate type for `id`)

        ),

    ],

    billing_mode="PAY_PER_REQUEST", # On-demand billing mode

    tags={

        "Environment": environment,

        "Created_By": "Pulumi"

    }

)

```

Create an IAM Role for the Lambda function

Create Lambda execution role

```

lambda_role = aws.iam.Role(

    "lambdaExecutionRole",

    assume_role_policy=json.dumps({

        "Version": "2012-10-17",

        "Statement": [{

            "Action": "sts:AssumeRole",

```

```

        "Principal": {
            "Service": "lambda.amazonaws.com"
        },
        "Effect": "Allow",
        "Sid": ""
    }]
})
)

```

Create inline policy for the role

```

dynamodb_policy = aws.iam.RolePolicy(
    f"lambdaRolePolicy-{environment}",
    role=lambda_role.id,
    policy=pulumi.Output.json_dumps({
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Action": [
                    "dynamodb:Scan",
                    "dynamodb:PutItem",
                    "dynamodb:GetItem",
                    "dynamodb:UpdateItem",
                    "dynamodb:DeleteItem",
                    "dynamodb:Query"

```

```

    ],

    "Resource": [

        dynamodb_table.arn,

        pulumi.Output.concat(dynamodb_table.arn, "/*")

    ]

},

{

    "Effect": "Allow",

    "Action": [

        "logs:CreateLogGroup",

        "logs:CreateLogStream",

        "logs:PutLogEvents"

    ],

    "Resource": "arn:aws:logs:*:*:*"

}

]

}))

)

```

Create a Lambda function using the Docker image

```

lambda_function = aws.lambda_.Function(

    f"my-serverless-function-{environment}",

    role=lambda_role.arn,

    package_type="Image",

    image_uri=docker_image,

```

```

memory_size=512,

timeout=30,

opts=pulumi.ResourceOptions(depends_on=[lambda_role])

)

# Create an API Gateway REST API

api = aws.apigateway.RestApi(f'my-api-{environment}',

    description="My serverless API")

# Create a catch-all resource for the API

proxy_resource = aws.apigateway.Resource(f'proxy-resource-{environment}',

    rest_api=api.id,

    parent_id=api.root_resource_id,

    path_part="{proxy+}")

# Create a method for the proxy resource that allows any method

method = aws.apigateway.Method(f'proxy-method-{environment}',

    rest_api=api.id,

    resource_id=proxy_resource.id,

    http_method="ANY",

    authorization="NONE")

# Integration of Lambda with API Gateway using AWS_PROXY

integration = aws.apigateway.Integration(f'proxy-integration-{environment}',

    rest_api=api.id,

```

```

resource_id=proxy_resource.id,

http_method=method.http_method,

integration_http_method="POST",

type="AWS_PROXY",

uri=lambda_function.invoke_arn) # Ensure lambda_function is defined

```

```

lambda_permission = aws.lambda_.Permission(f'api-gateway-lambda-permission-
{environment}',

    action="lambda:InvokeFunction",

    function=lambda_function.name,

    principal="apigateway.amazonaws.com",

    source_arn=pulumi.Output.concat(api.execution_arn, "/*/*")

)

```

Deployment of the API, explicitly depends on method and integration to avoid timing issues

```

deployment = aws.apigateway.Deployment(f'api-deployment-{environment}',

    rest_api=api.id,

    stage_name="dev",

    opts=pulumi.ResourceOptions(

        depends_on=[method, integration, lambda_permission] # Ensures these are created
        before deployment

    )

)

```

Output the API Gateway stage URL

```

api_invoke_url = pulumi.Output.concat(

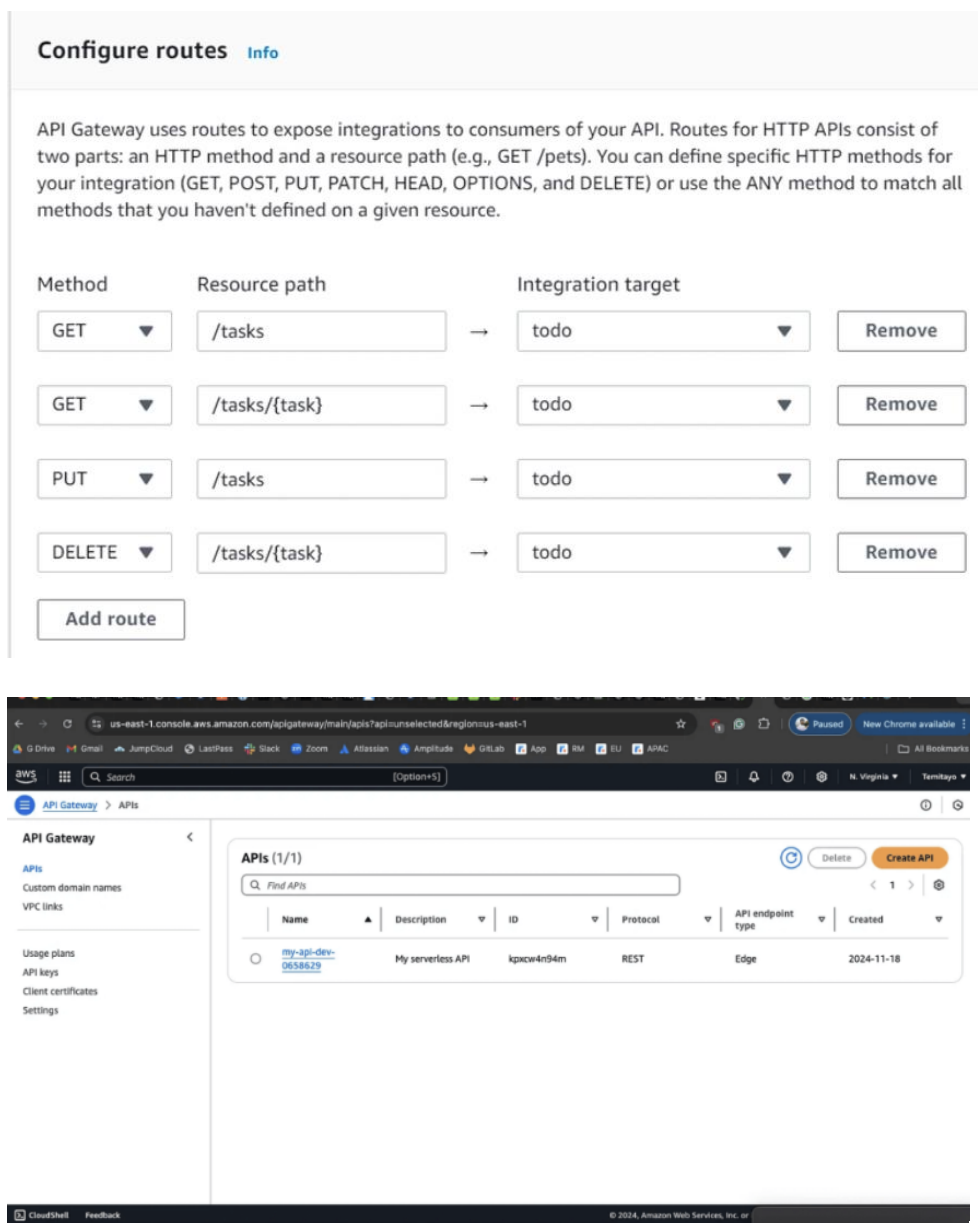
    "https://",    api.id,    ".execute-api.",    "us-east-1",    ".amazonaws.com/",
    deployment.stage_name

)

pulumi.export("api_invoke_url", api_invoke_url)

```

Screenshots to include:



GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/health

Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (12) Test Results

Status: 200 OK Time: 1837 ms Size: 605 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Everything looks good!"
3 }
```

GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos

Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (12) Test Results

Status: 200 OK Time: 189 ms Size: 684 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "b56c1dbb-b3d8-4885-ac59-6885370ba0e0",
3   "text": "Testing endpoint",
4   "completed": true,
5   "timestamp": 1732002518
6 }
7
8
```


You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- POST https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- PATCH https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- PATCH https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- POST https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- POST https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos
- GET https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

POST https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos

Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "text": "Testing endpoint", // The text/description of the todo
3   "completed": true // Whether the todo is completed or not
4 }
5
```

Body Cookies Headers (12) Test Results

Status: 201 Created Time: 178 ms Size: 687 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "b56c1dbb-b3d8-4885-ac59-6885370ba0e0",
3   "text": "Testing endpoint",
4   "completed": true,
5   "timestamp": 1732002518
6 }
```

https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos/b56c1dbb-b3d8-4885-ac59-6885370ba0e0

Save

PATCH https://kpxcw4n94m.execute-api.us-east-1.amazonaws.com/dev/todos/b56c1dbb-b3d8-4885-ac59-6885370ba0e0

Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "text": "Testing endpoint", // The text/description of the todo
3   "completed": false // Whether the todo is completed or not
4 }
5
```

Body Cookies Headers (12) Test Results

Status: 200 OK Time: 128 ms Size: 683 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "b56c1dbb-b3d8-4885-ac59-6885370ba0e0",
3   "text": "Testing endpoint",
4   "completed": false,
```

ADD TODO

There are currently no todos available to display

ADD TODO

- Play around with Serverless [Delete](#)
- Create a Todo application [Delete](#)

CONCLUSION

This project, titled "**Serverless Personal To-Do List Application**," offers a comprehensive exploration into the capabilities and benefits of **serverless computing using Amazon Web Services (AWS)**. The successful implementation of this project showcases how cloud-native technologies can be leveraged to build scalable, cost-efficient, and highly available applications without the need to manage physical servers or complex infrastructure.

The key idea behind this project was to **build a fully functional CRUD (Create, Read, Update, Delete) application** for personal task management that relies entirely on a serverless backend. The core AWS services used were:

- **Amazon API Gateway** – to define and expose RESTful APIs.
- **AWS Lambda** – to execute backend logic in response to API calls.
- **Amazon DynamoDB** – to store to-do items in a highly available and scalable NoSQL database.
- **AWS SAM (Serverless Application Model)** – to define the infrastructure as code and manage deployments in a version-controlled and repeatable way.

Learnings and Achievements

This project has provided **invaluable hands-on experience** with the tools, frameworks, and concepts used in real-world serverless application development. Here are the major takeaways from the implementation:

1. Understanding Serverless Architecture

The project enabled a deep understanding of **event-driven architectures**, particularly how functions (AWS Lambda) are triggered by events such as HTTP requests routed via API Gateway. The idea of not having to provision or manage infrastructure simplifies development and allows developers to focus entirely on the **business logic** of their application.

We explored how **Lambda abstracts server management**, enabling auto-scaling, fault tolerance, and high availability by default. This approach reduced operational overhead and cost, especially for a lightweight application like a personal to-do list app.

2. Real-World Application of AWS Services

One of the most valuable aspects of this project was learning to use a combination of **AWS cloud services** together:

- **API Gateway** was used to define secure and structured endpoints (`/todo`, `/todo/{id}`) for all CRUD operations.
- **Lambda functions** handled incoming requests and performed operations on DynamoDB.
- **DynamoDB** was used as the backend data store due to its scalability, low latency, and fully managed nature.
- **IAM Roles** and **permissions** were configured to ensure secure access to AWS services.

This provided a **realistic, end-to-end experience** of how cloud-native microservices are designed and built in the industry.

3. Practical Exposure to CRUD and RESTful APIs

Implementing CRUD operations gave a better understanding of REST principles, data modeling, and API design patterns. Each operation (Create, Read, Update, Delete) was mapped to an HTTP method (POST, GET, PUT, DELETE), and appropriate Lambda logic was created for each.

- We practiced **handling JSON request bodies, parsing API Gateway event objects, and responding with structured JSON responses**.
- Each response was tested using tools like **Postman**, and logs were analyzed using **AWS CloudWatch**, further strengthening debugging skills.

4. Infrastructure as Code with AWS SAM

Learning to use **AWS SAM templates** (`template.yaml`) helped understand how to manage infrastructure in a repeatable and version-controlled manner. This aligned with modern DevOps practices, where infrastructure is treated just like code and can be deployed or rolled back with ease.

SAM also made it easier to manage API routes, permissions, Lambda configurations, and environment variables—all in a declarative way.

We also used:

- `sam build` – to compile dependencies.
- `sam deploy --guided` – for step-by-step deployment to AWS.

Scalability and Future Enhancements

Although the current project implementation provides a working application for personal task management, it is built on **AWS's highly scalable serverless stack**, which means it can support **hundreds or even millions of users** without changes to the core architecture.

Potential future enhancements include:

1. User Authentication with AWS Cognito

By integrating **Amazon Cognito**, user accounts and authentication flows (sign-up, login, etc.) can be added. This will allow multiple users to maintain **private to-do lists**, adding security and personalization.

2. Frontend with React + AWS Amplify

A beautiful and responsive frontend could be built using **React**, while **AWS Amplify** could simplify integration with the backend APIs, authentication, and hosting. This will give the application a **complete full-stack experience**, making it usable via web and mobile interfaces.

3. Analytics and Monitoring with AWS CloudWatch

For deeper insights into how users interact with the application, **CloudWatch Insights** and **custom metrics** can be added. This would help monitor performance, user engagement, error rates, and function invocations.